

Autonomous Lottery Ticket Discovery — 实验执行表

0. 总体实验原则

整个项目分成四个阶段：

Stage	核心问题	实验
A	Compressibility Frontier 是否真实存在？	E0-E3
B	Adaptive Ticket Search 是否优于固定剪枝？	E4-E7
C	Compression Frontier 数据是否能帮助恢复？	E8-E11
D	模型能否形成长期 Autonomous Self-Compression？	E12-E14

前一阶段没有稳定信号，则暂缓进入下一阶段。

Stage A：先验证 Compressibility Frontier

E0. Dense Model Baseline

项目	设置
Experiment ID	E0
目的	建立所有后续实验的 dense baseline
Model	Qwen2.5-3B-Instruct
Compression	None
Evaluation	PPL + Math + Knowledge + Reasoning + Instruction + Code
Seeds	1
GPU	1×24GB
输出	Dense performance vector $\mathbf{P}(M_0)$
成功条件	所有 benchmark pipeline 可稳定复现
预计优先级	P0

记录：

$$\mathbf{P}(M_0) = [PPL, Math, Knowledge, Reasoning, Instruction, Code].$$

以及：

- GPU memory ；
- latency ；
- parameter count ；
- model size。

后续所有 acceptance threshold 都以 E0 为基准。

E1. One-shot Sparsity Curve

项目	设置
Experiment ID	E1
目的	判断是否存在明显 performance cliff
Model	Qwen2.5-3B
Method	Wanda
Sparsity	10%, 20%, 30%, 40%, 50%, 60%, 70%
Recovery	None
Seeds	1
GPU	1×24GB
关键变量	sparsity
核心图	performance vs sparsity
成功条件	存在明显非线性 degradation / capability-specific degradation
优先级	P0

得到：

$$P_j(s)$$

其中 j 是不同 capability。

重点不是平均性能，而是观察：

$$s_{\text{math}}^*, s_{\text{knowledge}}^*, s_{\text{code}}^*, \dots$$

是否不同。

希望观察

例如：

Sparsity	Math	Knowledge	Reasoning
20%	-0.3	-0.1	-0.2
40%	-1.2	-0.8	-0.9
50%	-4.5	-1.5	-3.0
60%	-12.0	-4.0	-10.0

如果不同能力出现不同 cliff：

Capability-specific Compressibility Frontier

成立。

E2. Iterative vs One-shot Compression

项目	设置
Experiment ID	E2
目的	验证逐步压缩是否优于一次性压缩
Model	Qwen2.5-3B
Method	Wanda
Target sparsity	40%, 50%, 60%
Baseline A	One-shot
Baseline B	5% incremental pruning
Recovery	None
Seeds	3
GPU	1-2×24GB
核心指标	Final performance
成功条件	iterative 稳定优于 one-shot
优先级	P0

比较：

$$M_0 \rightarrow M_{50\%}$$

vs.

$$M_0 \rightarrow M_5 \rightarrow M_{10} \rightarrow \cdots \rightarrow M_{50}.$$

如果没有差异，说明：

“逐代 lineage”本身可能没有价值，

后面的 Lottery Ticket 叙事需要重新考虑。

E3. Compression-Induced Capability Gap

项目	设置
Experiment ID	E3
目的	验证 Compression Gap 是否能描述真实能力损失
Parent	Dense model
Children	30%, 40%, 50%, 60% sparse
Candidate Pool	2k-5k prompts
Score	$G(x) = L_S(y_T x) - L_T(y_T x)$
Evaluation	downstream success/failure
Seeds	1
GPU	1×24GB
核心指标	correlation / AUROC
成功条件	Gap 显著预测 downstream degradation
优先级	P0

计算：

$$corr(G(x), Failure(x)).$$

同时比较：

$$G(x)$$

vs.

- teacher NLL ;
- student NLL ;
- prompt length ;
- random score。

Go 条件

希望至少满足：

$$G(x)$$

明显优于：

$$L_S(x)$$

预测 compression-specific failure。

否则前面 Compression Frontier 的核心 formulation 需要修改。

Stage B : Adaptive Lottery Ticket Search

E4. Fixed Incremental Schedule

项目	设置
Experiment ID	E4
目的	建立 iterative pruning baseline
Initial	Dense
Step	+5% sparsity
Region selection	Wanda importance
Recovery	None
Stopping	predefined threshold
Seeds	3
核心输出	Final reachable sparsity
优先级	P0

Acceptance :

$$Perf(M_t) \geq Perf(M_0) - \delta.$$

例如 :

$$\delta_{aggregate} = 2\%$$

并设置 :

$$\delta_j = 5\%$$

防止单一能力崩溃。

最终得到 :

$$s_{max}^{fixed}.$$

E5. Adaptive Pruning Ratio

项目	设置
Experiment ID	E5
目的	验证是否值得自适应决定压缩步长
Action	+3%, +5%, +8%
Region	固定 importance-based
Controller	heuristic
History	有
Recovery	None
Seeds	3
Baseline	E4
成功条件	相同 quality 下达到更高 compression
优先级	P1

Controller 根据上一轮 :

$$\Delta Perf_t$$

决定 :

```
loss small → +8%
loss medium → +5%
loss large → +3%
```

先不用 LLM。

这是为了回答：

adaptive schedule 本身有没有价值？

E6. Adaptive Region Selection

项目	设置
Experiment ID	E6
目的	验证 capability feedback 是否帮助选择剪枝位置
Candidate regions	layer groups
Ratio	固定 5%
Baseline A	global Wanda
Baseline B	structural sensitivity
Ours	structural + capability gap
Seeds	3
优先级	P1

例如候选：

```
MLP 0-7
MLP 8-15
MLP 16-23
Attention 0-7
Attention 8-15
...
```

策略输入：

$[StructuralSensitivity, CapabilityGap]$.

核心问题

CapabilityFeedback

是否比纯 weight / activation sensitivity 提供额外价值？

E7. Reversible Ticket Search

项目	设置
Experiment ID	E7
目的	验证 rewind + regrow 是否改善 frontier
Baseline	monotonic pruning
Ours	prune + rollback + regrow
Candidate tickets	K=3
Seeds	3
Recovery	None
关键指标	reachable sparsity / rollback count
优先级	P1

每代产生：

$T_t^1, T_t^2, T_t^3.$

例如：

A: prune MLP 8-14 by 5%

B: prune ATT 3-8 by 7%

C: regrow MLP 19 + prune MLP 5 by 8%

主要比较

$s_{max}^{reversible}$

vs.

$s_{max}^{monotonic}.$

如果 regrowth 没有收益，可在最终方法中删除。

Stage C : Compression Frontier Recovery

E8. Recovery Data Baseline

项目	设置
Experiment ID	E8
目的	证明“什么数据用于恢复”很重要
Parent	Dense
Child	50% sparse
Recovery	LoRA 200–500 steps
Data size	256 / 512 / 1k
数据	Random
Seeds	3
优先级	P0

记录：

$$RecoveryGain = Perf(M_{recover}) - Perf(M_{compressed}).$$

作为后续数据方法 baseline。

E9. Hard Data vs Compression Gap Data

这是前面 Compression Frontier 方案最重要的实验。

数据策略	定义
Random	随机
Teacher Hard	High L_T
Student Hard	High L_S
Low Gap	Low $L_S - L_T$
High Gap	High $L_S - L_T$
High Gap + Random	80/20 exploitation/exploration

固定：

- 相同 number of examples ；
- 相同 total token budget ；
- 尽量 matched length distribution ；
- 相同 recovery steps。

希望看到

$$HighGap > StudentHard > Random > LowGap.$$

真正关键的结果是：

$$HighGap > StudentHard.$$

这说明：

$CompressionSpecificDifficulty /$

E10. Compression-specificity Matrix

项目	设置
Experiment ID	E10
目的	验证 recovery/calibration data 是否 compression-policy specific
Method A	Wanda
Method B	SparseGPT
Optional	AWQ / GPTQ
数据	D_A, D_B
Seeds	3
优先级	P1

做交叉矩阵：

Recovery data	Wanda Child	SparseGPT Child
Random	Base	Base
D_{Wanda}	?	?
$D_{SparseGPT}$	?	?

希望：

$$Perf(Wanda, D_{Wanda}) > Perf(Wanda, D_{SparseGPT})$$

且：

$$Perf(SparseGPT, D_{SparseGPT}) > Perf(SparseGPT, D_{Wanda}).$$

如果成立，可以强力支持：

$$D^* = f(M, C).$$

E11. Synthetic Frontier Data

项目	设置
Experiment ID	E11
目的	验证 BigBang-style generation 是否带来额外收益
Seed data	Top compression gap examples
Generator	Parent model / stronger model
Generation	failure abstraction → new prompts
Critic	competence + gap + verifiability + diversity
Recovery	LoRA / KD
Seeds	3
优先级	P1

比较：

Random
Student Hard
High Gap Real
Paraphrase High Gap
Synthetic Frontier
Oracle

特别加入：

Paraphrase baseline

非常重要。

否则 reviewer 会问：

你只是扩写了 High-Gap data。

所以：

$$SyntheticFrontier > Paraphrase$$

才证明 generator 学到了 failure structure。

两种成功形式都可以接受

Accuracy gain

$$Synthetic > HighGap.$$

或者：

Data efficiency

达到相同恢复性能：

$$|D_{syn}| < |D_{real}|.$$

Stage D : Full Autonomous Lottery Ticket Discovery

E12. Controller Ablation

项目	设置
Experiment ID	E12
目的	判断是否真的需要 LLM controller
Controller A	Fixed heuristic
Controller B	Rule-based adaptive
Controller C	LLM controller
Inputs	profile + history
Action	ticket proposal

项目	设置
Seeds	3
优先级	P1

LLM controller 必须和简单 heuristic 比。

否则：

“LLM 可以根据日志做 pruning”

本身不构成贡献。

希望：

$$Frontier_{LLM} > Frontier_{heuristic}$$

或者在相同 frontier 下：

$$SearchCost_{LLM} < SearchCost_{heuristic}.$$

E13. External Controller vs Self Controller

这是整个项目最关键的实验之一。

External

始终：

$$Controller = M_0.$$

Self

Generation t ：

$$Controller = M_t.$$

指标	External	Self
Final sparsity		
Benchmark score		
Search trials		
Rollbacks		

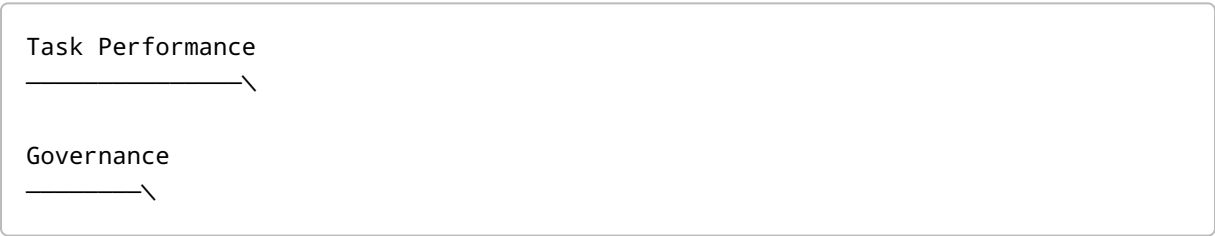
指标	External	Self
Governance accuracy		
GPU cost		

研究：

$$Gov(M_t)$$

随 compression 的变化。

可能出现：



即：

$$GovernanceFrontier < TaskFrontier.$$

如果成立，这可能本身就是一个非常好的 research finding。

E14. Full Autonomous Run

最终完整实验。

项目	设置
Experiment ID	E14
Model	Qwen 3B → 后续 7B
Duration	10-30 generations
Ticket candidates	3-5 / generation
Actions	prune / regrow / recover
Data	compression-frontier synthetic data
Controller	current accepted model
Stop	autonomous

项目	设置
Seeds	3
优先级	P2

每一代：

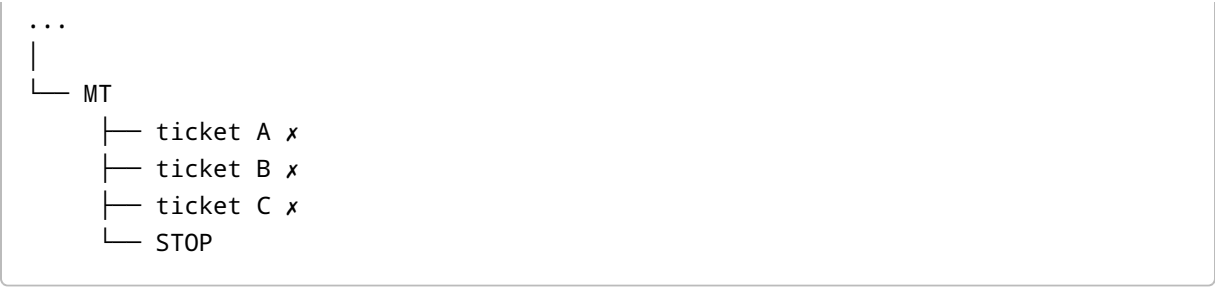
1. Profile M_t
2. Propose K tickets
3. Cheap evaluation
4. Select candidates
5. Compress
6. Measure compression gap
7. Generate recovery data
8. Recover
9. Full evaluation
10. Accept / rewind
11. Update memory
12. Stop / next generation

最终输出 lineage：

```

M0
|
├─ T0-A x
|
├─ T0-B ✓ → M1
|
|
├─ T1-A x
├─ T1-B x
├─ T1-C ✓ → M2
|

```



最终的：

$$M_T$$

就是系统发现的：

Empirical Compressibility Frontier

31. 整体实验清单

ID	实验	核心结论	优先级
E0	Dense baseline	建立评价基准	P0
E1	One-shot sparsity curve	Frontier 是否存在	P0
E2	One-shot vs iterative	逐步搜索是否必要	P0
E3	Compression Gap	Gap 是否预测损伤	P0
E4	Fixed iterative	Autonomous baseline	P0
E5	Adaptive ratio	自适应步长是否有效	P1
E6	Adaptive region	Capability feedback 是否有效	P1
E7	Rewind + regrow	可逆搜索是否有效	P1
E8	Random recovery	Recovery baseline	P0
E9	High-gap recovery	Compression-specific data	P0
E10	Cross-compression	Policy-specific frontier	P1
E11	Synthetic frontier	BigBang-style generator	P1
E12	Controller comparison	LLM controller 是否必要	P1
E13	Self vs external	Self-governance frontier	P1
E14	Full autonomous run	Autonomous Lottery Ticket	P2

32. 最关键的 Go / No-Go Gates

Gate A — E1/E2

如果不存在：

- performance cliff；
- capability-specific frontier；
- iterative advantage；

则暂时不要做 Agent。

Gate B — E3/E9

如果：

CompressionGap

既不能预测 degradation，也不能指导 recovery：

则放弃 Compression Frontier 数据主线。

Gate C — E5/E6/E7

如果 adaptive search：

≠ fixedschedule,

则不用做复杂 controller。

Gate D — E11

如果：

SyntheticFrontier ≤ HighGapReal,

则 synthetic generation 只作为 optional module，不作为论文核心。

Gate E — E13

如果 compressed child 无法可靠控制自己的下一轮：

不要强行声称完全 self-compressing。

可以将方法调整为：

PersistentMetaController + AutonomousTicketSearch.

同时把：

Self-GovernanceFrontier

作为重要分析结论。

33. GPU 执行建议

Phase 1：E0–E3

1 × RTX 4090 24GB

完全够。

主要跑：

- Qwen 3B；
- 2k–5k profiling pool。

Phase 2：E4–E11

推荐：

2–4 × 4090

因为不同 ticket / seed 可以并行。

Phase 3 : E12-E14

推荐：

4 × 4090

或者：

2-4 × A100/L40S

主要需求来自：

- 多 ticket parallel exploration ；
- recovery training ；
- benchmark evaluation。

不是单个模型装不下，而是增加实验 throughput。

34. 最先应该启动的六个实验

不要一口气实现 E0-E14。

第一批只启动：

E0 Dense baseline
E1 sparsity curve
E2 one-shot vs iterative
E3 Compression Gap correlation
E8 random recovery
E9 High-Gap vs Student-Hard vs Random

这六个实验回答整个项目最核心的两个科学问题：

Q1

Is there a meaningful Compressibility Frontier?

Q2

Can the model diagnose and repair compression-specific failures?

只有这两个问题答案都是 **Yes**，才值得投入资源做：

AdaptiveTicketSearch $\rightarrow$ *SyntheticGeneration* $\rightarrow$ *SelfController*.

换句话说，**第一阶段甚至完全不需要 Agent**。